23/9/2026

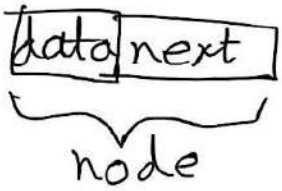
Final ৭ minimum 10 mark ~~50~~ সর্বোচ্চ 15. linked list

24/9/2026

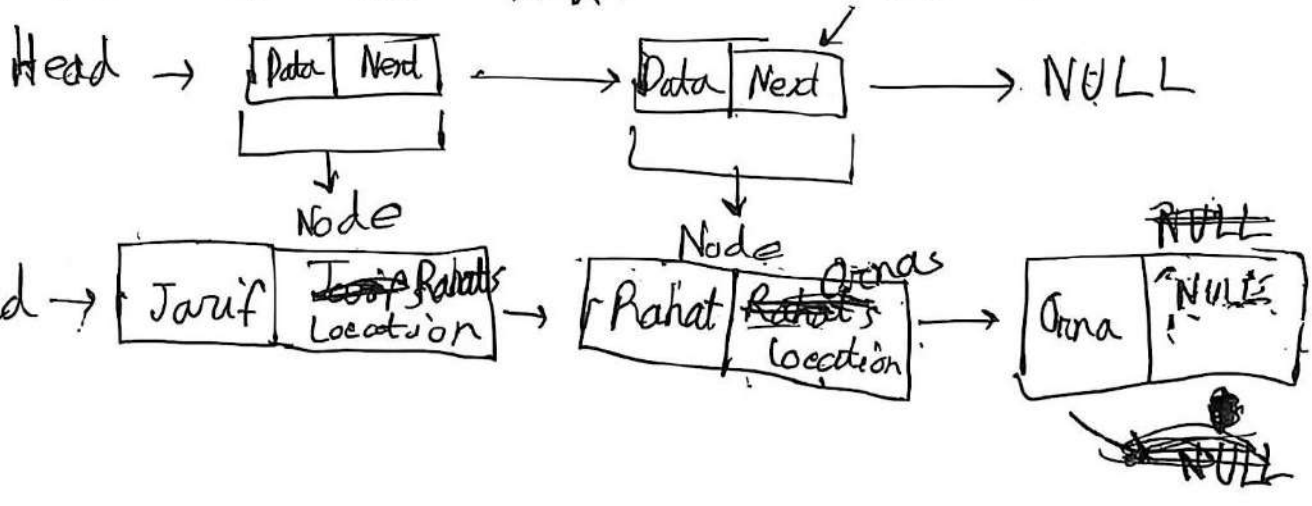
Linked List

Linear data structure

- A linked list is an ordered collection of finite, homogeneous data elements called nodes where the linear order is maintained by means of links or pointers.
- Each element is represented by a node



- Each node is divided into two parts:
  1. Data part contains the information of element
  2. Link part contains the address of the next node in the link.



- Linked list has a pointer variable, Head or start containg address of first node
- If start contains NULL value, it means the list is empty
- The last node contains NULL reprosting invalid

Dynamic representation

Terminologies

Malloc()

```
#include <stdlib.h>
int* ptr = (int*) malloc (5*sizeof(int));
```

4 byte

optional

ptr = 20 bytes of memory

Structure

```
struct node
{
    int data;
    struct node *next;
};
```

Int ~~ptr~~ ~~ptr~~ ~~ptr~~ !

~~ptr~~ Pointer self refrence

5000 → 5011 → arrow operator

ptr = malloc (sizeof (struct node))

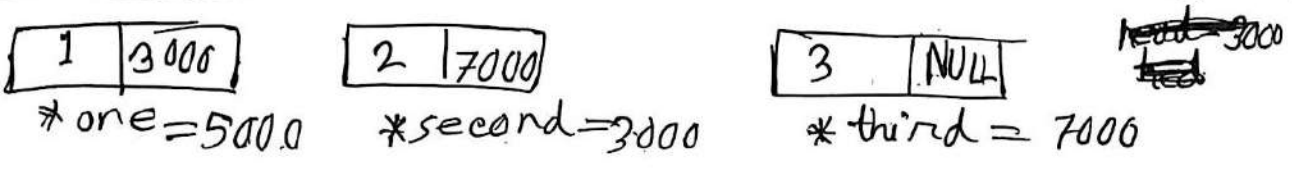
```
struct node *head;
struct node *one = NULL;
struct node *two = NULL;
struct node *three = NULL;

one = malloc (sizeof (struct node));
two = "";
three = ""

one->data = 1;
two->data = 2;
three->data = 3;

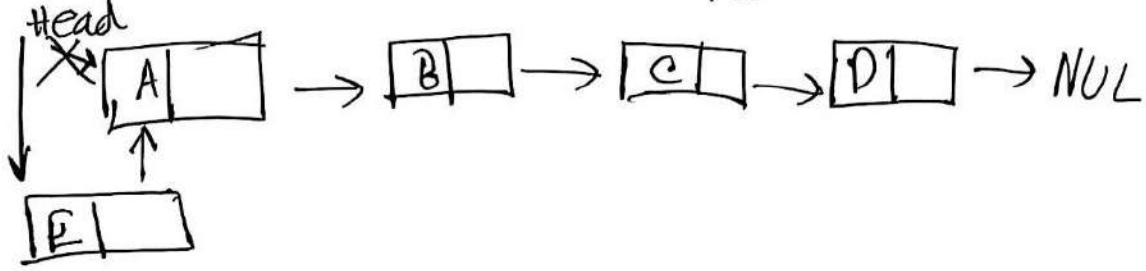
one->next = two;
two->next = three;
three->next = NULL;

head = one;
```



3409/2026

Add a new node at front



Algorithm: InsertAtHead (head, data)

1. Create a new node NEW
2. Set NEW → data = data
3. set NEW → next = head
4. Update head = NEW
5. Return head

Time complexity : O(1)

* head = NULL



*new = malloc( ) 5000

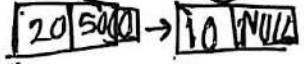
new → data = value;
10

new → next = head

head = new

---

① head = 5000 ②

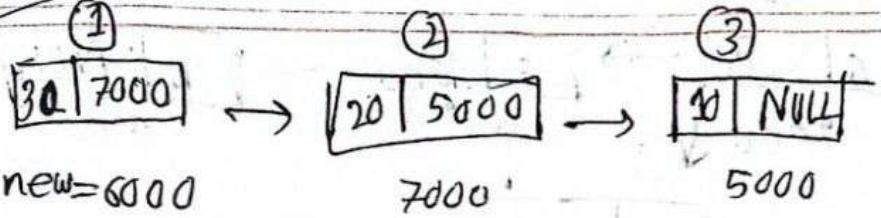


*new = 7800

new → data = value = 20

new → next = head = 5000 head = new

3rd time calling

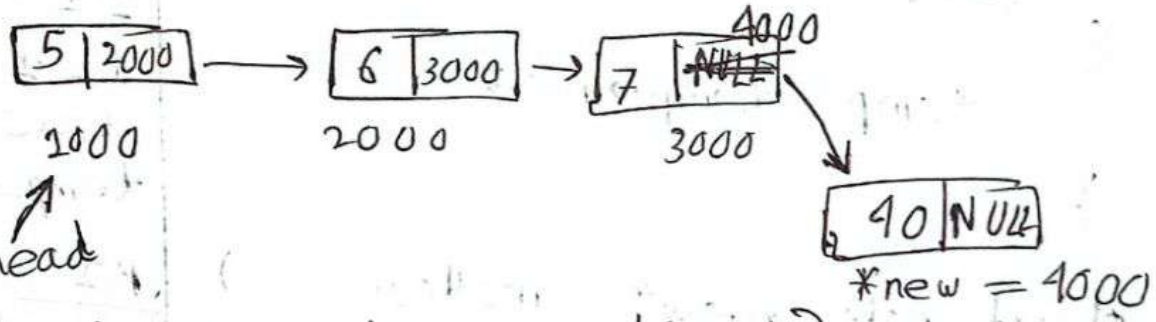


new → data = value = 30

new → next = head = 7000

head = new

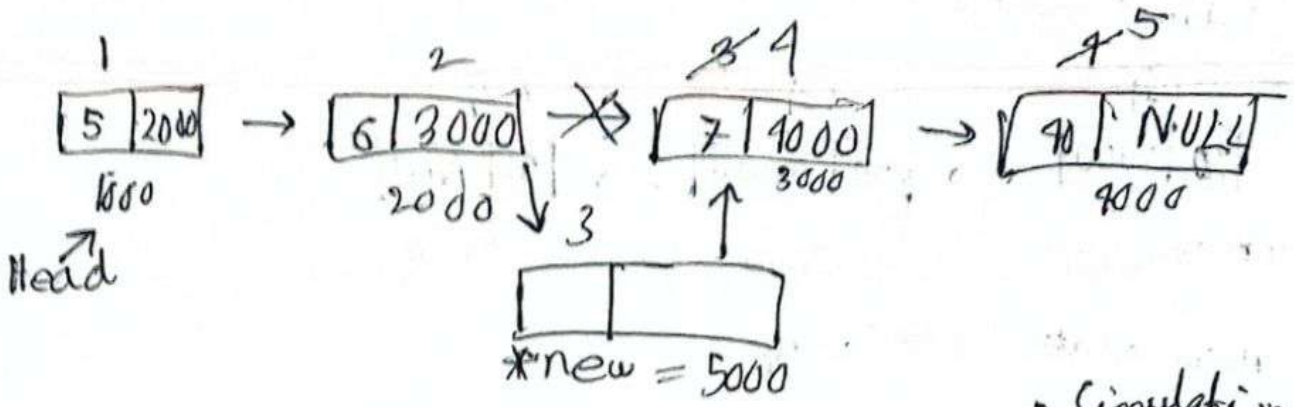
~~Algorithm~~: Insert at last




```
struct node *temp = head;
while(temp→next != NULL)
{
    temp = temp→next;
}
```

যুদি কোনো temp Last node এ থাকবে

```
struct node *new = malloc(.....)
    new→data = value; //40
    new→next = NULL;
    temp→next = new;
```




```
struct node *temp = head;
int pos;
scanf("%d", &pos);
for (int i=1; i<pos-1; i++)
{
    temp = temp->next;
}
struct node new* = malloc(---)
new->data = value
new->next = temp->next
temp->next = new
```

important lines

Simulation

temp = 1000

Round 1

i = 1

i < pos-1

1 < 3-1

1 < 2

true

temp = 2000

R2

i = 2

2 < pos-1

2 < 3-2

2 < 2

false

for loop থেকে বের হয়ে যাবে

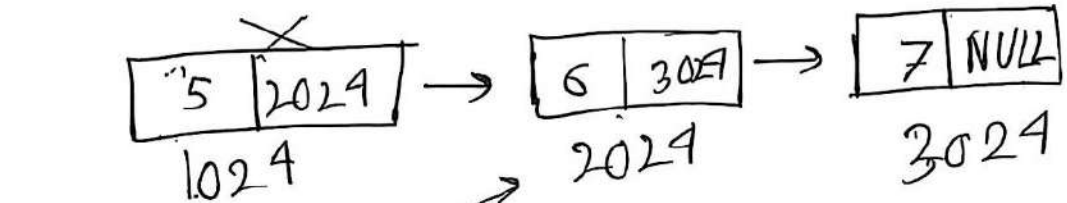
temp থাকবে 2000

01/10/26

free() function দিয়ে আমরা memory release করতে পারি (Malloc এর ক্ষেত্রে)

Deletion

Deletion from head



head

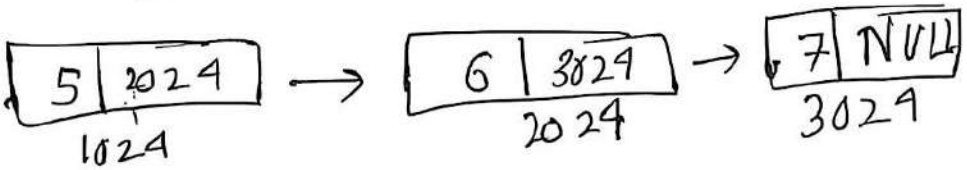
struct node *temp = head; //temp=1024

free(head)

head = head -> next

free(temp);

Deletion from last -



head

```
struct node *temp = head, *prev;
while (temp->next != NULL) {
    prev = temp;
    temp = temp->next;
}
free(temp);
prev->next = NULL;
```

Simulation

temp = 1024

R1

2024 != NULL
prev = 1024;
temp = 2024

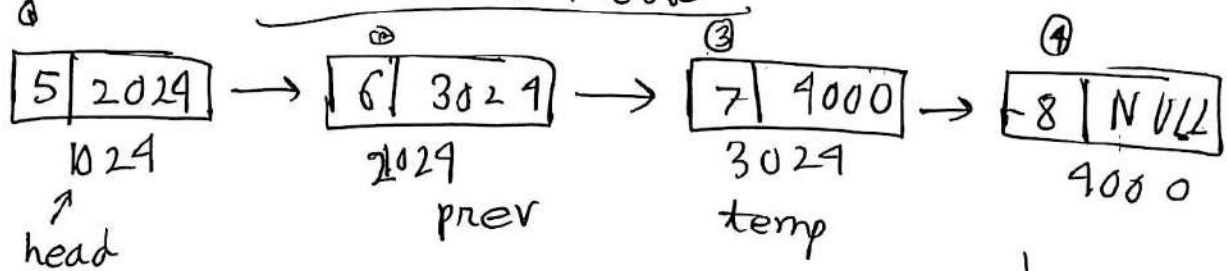
R2

3024 != NULL
prev = 2024;
temp = 3024

R3

NULL != NULL

Delete from middle



③ Delete 4th

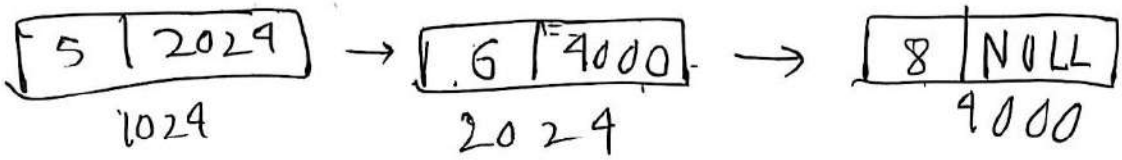
```
int pos;
scanf("%d", &pos); // 3
struct node *temp = head, *prev;
for(int i=1; i<pos; i++)
{
    prev = temp;
    temp = temp -> next;
}
prev -> next = temp -> next
free(temp);
```

temp = 1029
prev = ?

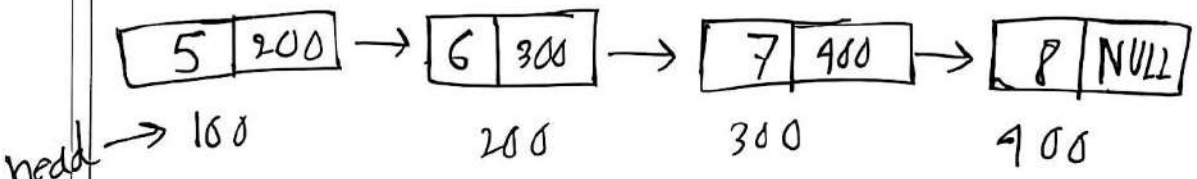
R1
i = 1
i < pos
1 < 3
prev = 1029
temp = 2029

R2
i = 2
2 < 3
prev = 2029
temp = 3029

R3
i = 3
3 < 3 ✗



Search



key = 8

```
int pos = 0
struct node *temp = head;
while(temp != NULL)
{
    pos++;
    if (key == temp->data)
    {
        printf("Item found in pos %d", pos);
    }
    temp = temp->next;
}
```

temp = 100

R1
100 != NULL
8 == 5
temp = 200

R2
200 != NULL
8 == 6
temp = 300

R3
300 != NULL
8 == 7
temp = 400

R4
400
NULL != NULL
8 == 8
temp = NULL

R5
NULL != NULL

Circular singly Linkedlist

```
struct node *temp = head;
while (temp -> next != NULL)
{ temp = temp -> next;
}
temp -> next = head;
```

7/10/2026

ODD No. Node ಇದ್ದ Linked list ದಲ್ಲಿ middle node delete

```
void deletmiddle()
{ struct node *temp = head, *prev;
  int n=0;
     while(temp != NULL) {
          n++;

          temp = temp -> next;

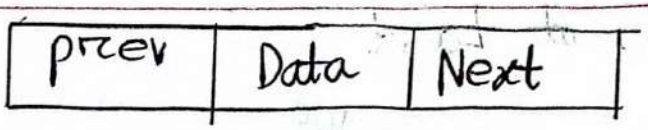
     }

  int m = n/2 + 1;
      temp = head;
  for(int i=1; i<m; i++)
  {       prev = temp;
          temp = temp -> next;

  }
  prev -> next = temp -> next
  free(temp);

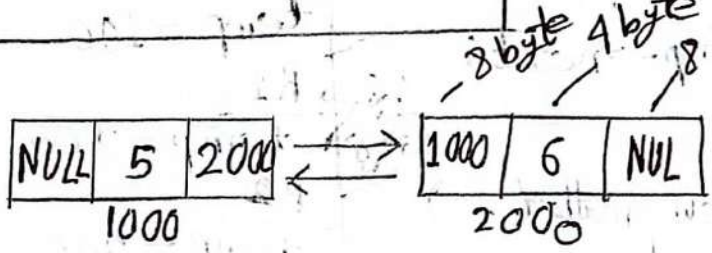
}
```

Doubly Linked List



Singly linke list - 12 byte
Doubly " " - 20 byte

```
struct node{
    int data;
    struct node *next;
    struct node *prev;
};
```



Initially *head = NULL

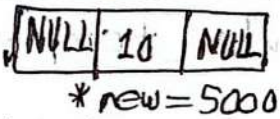
Insert at head

```
{
struct node *new = malloc(sizeof(struct node));
new->data = value;
new->next = head;
new->prev = NULL;
if(head != NULL){
head->prev = new;
}
head = new
}
```

Simulation

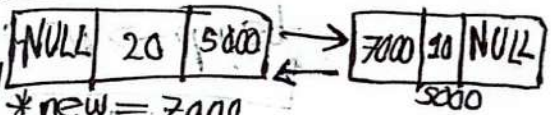
Initially
*Head = NULL

R1 ← Insert 1st time



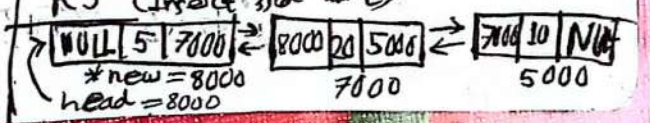
*new = 5000
head = 5000

R2 (Insert 2nd time)



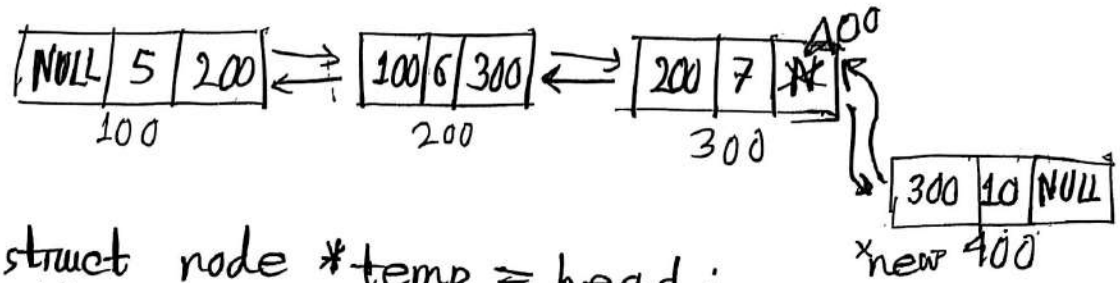
*new = 7000
head = 7000

R3 (Insert 3rd time)

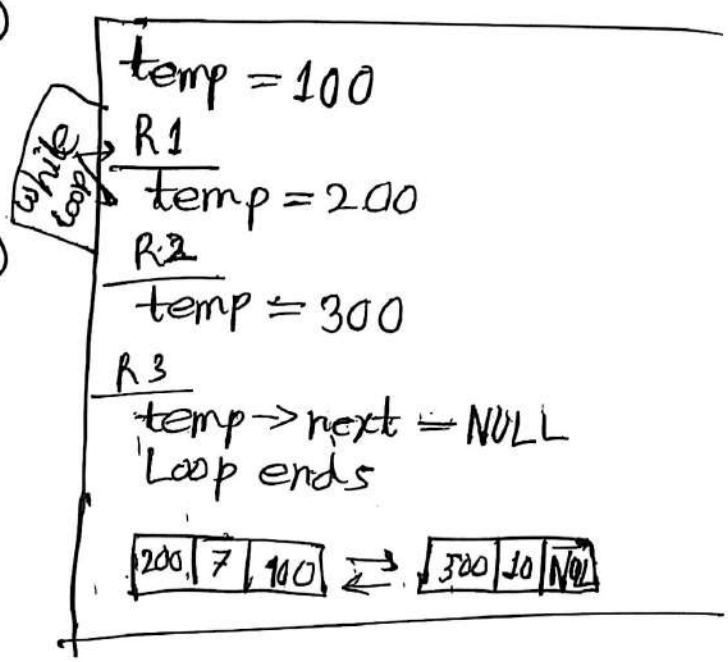


*new = 8000
head = 8000

Insertion at Last




```
struct node *temp = head;
while(temp->next != NULL)
{
    temp = temp->next;
}
struct node *new = malloc(..)
new->data = value;
new->next = NULL;
new->prev = temp;
temp->next = new;
```

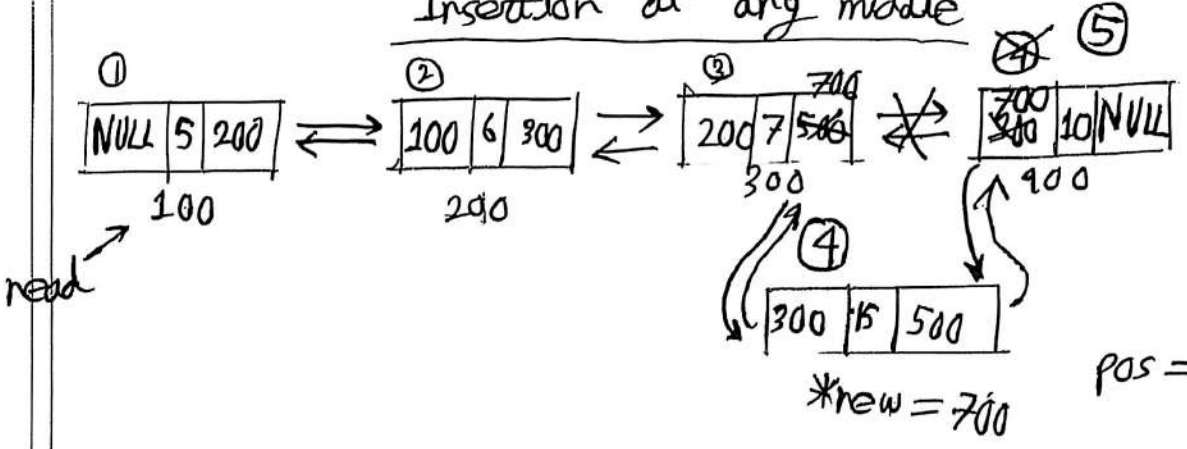


Delete at Last

```
struct node *temp = head;
while (temp->next != NULL)
{
    temp = temp->next;
}
temp->prev->next = NULL
free(temp);
```

300

Insertion at any middle




```
struct node *temp = head;
for (int i=1; i<pos-1; i++)
    temp = temp -> next;
}
struct node *new = malloc(...)
new -> data = value;
new -> next = temp -> next;
new -> prev = temp;
temp -> next = new;
new -> next -> prev = new;
```

500

simulation

Loop ⓘ temp = 100

R1 i = 1

temp = 200

R2 i = 2

temp = 300

R3 i = 3

pos+1 = 4-1 = 3

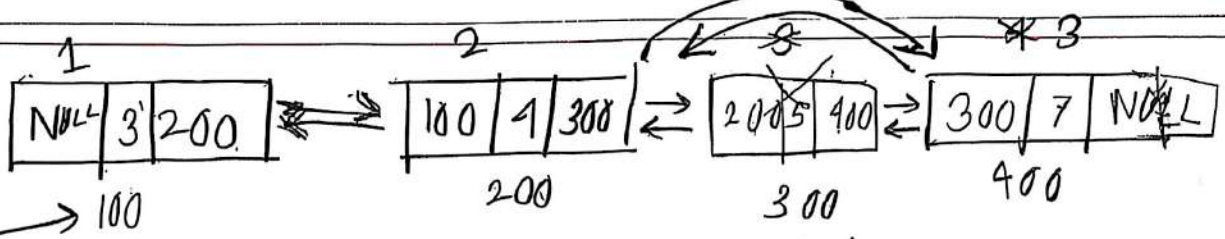
3 ✗ 3

Loop ends

∴ After loop, temp = 300

8/10/2026

Delete from any middle



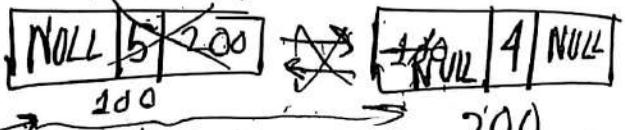
Head → 100

Delete(3)

```
struct node *temp = head;
for(int i=1; i<pos; i++)
{
    temp = temp->next;
}
temp->prev->next = temp->next;
temp->next->prev = temp->prev;
free(temp);
```

temp = 100
i = 1 < pos
R1 1 < 3
temp = 200
i = 2 < 3
R2
temp = 300
i = 3 ≮ 3

Delete from head

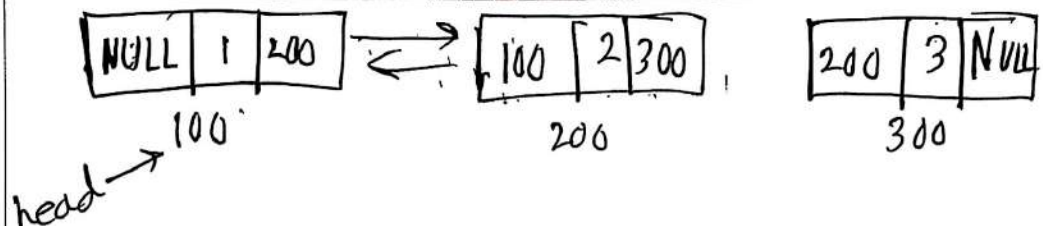


head →

```
struct node *temp = head; // temp = 100
head = head->next          // head = 200
head->prev = NULL;
free(temp);
```

Ring topology

Circular doubly linked list




```
struct node * temp = head;
while (temp -> next != NULL)
{ temp = temp -> next
}
```

temp = 100
R1
temp = 200
R2
temp = 300

```
temp -> next = head
head -> prev = temp
```

Last node ~~find~~ Loop

```
struct node * temp = head;
while (temp -> next != NULL){
        temp = temp -> next;
}
```

~~Ring topology~~

```
do {
printf("%d", temp->data);
temp = temp->prev;
while(temp != head)
```

```
while (temp != head) {
printf("%d", temp -> data);
temp = temp -> prev;
}
printf("%d", head -> data);
```

Solution by mam

```
struct node *temp = head;
while (temp -> next != NULL)
{  temp  = temp -> next;
}

while (temp != NULL)
{ printf("%d", temp->data);
  temp = temp -> prev;
}
```

Time complexity

Delet, Insert at head O(1)

Delete, Insertion at End O(n)

Delete, Insertion at specific location (Middle) O(n)

Useful

Space – Array তে আগাম space বরাদ্দ থাকে আমরা বরাদ্দ করে Linked list ও dynamically আমরা নেম থাকি সুবিধা

Online class - ৪ থেকে ৩টা ক্লাস (৩ ঘন্টা নিবে)

15-Oct-26 ১০ নাম্বার Must এসাইন

The right to Information act 2009

Setiaon - 2, 4, 6, 7, 8, 9, 24, 32 + Seedule

Setion2 [slide-3]

Section2 - [slid-5]

Information means any record or matrial - such as memos, books, maps, contracts, data, reports, accounts, photographs, audio, video, films, or any electronic or machine-readable documents - related to the structore, constitiv~~e~~ ~~authority~~.

বাস্তি vs নাগরিক

~~Sli~~

Setion 6 slide 7

~~authority এস~~

authority ক কোথায় কাকে কয় disclose করতে হবে।
annual report বলে ~~আলাদা~~ Report কয় কয়। CSS-এ
কোন গাছে কয়া লিখ বুঝিয়ে

slide দেখে বুঝিয়ে দেয়া লাগবে।

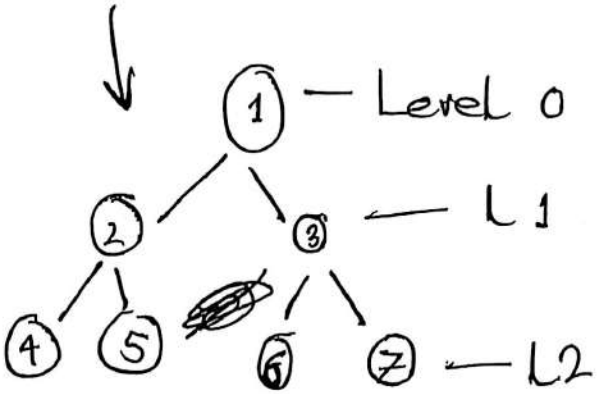
Section 8 -

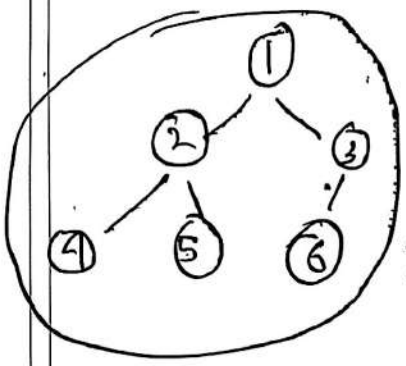
Section 9

Online
17-10-26

Heap

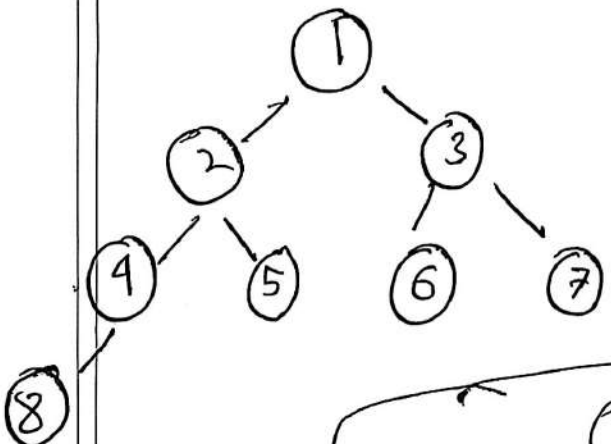
Compelete Binary tree (CBT)



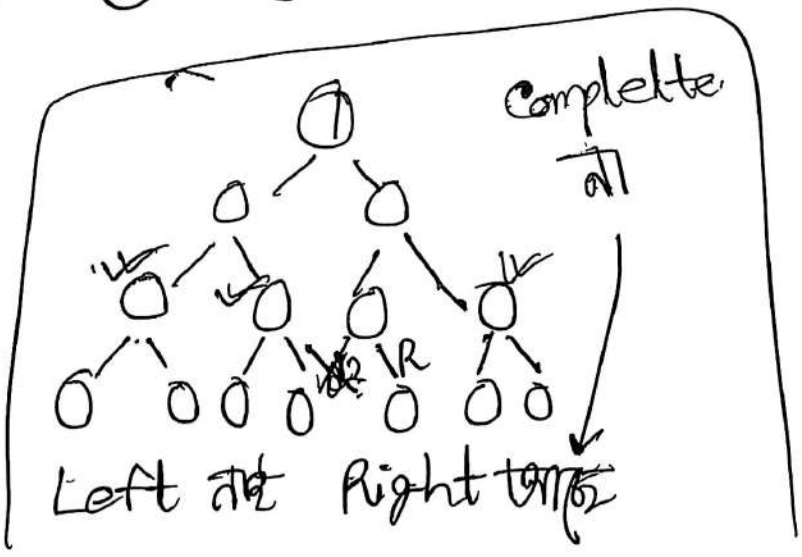


complelet

Last Level এ ডান দিকে কোনটা Absent থাকতে পারবে।

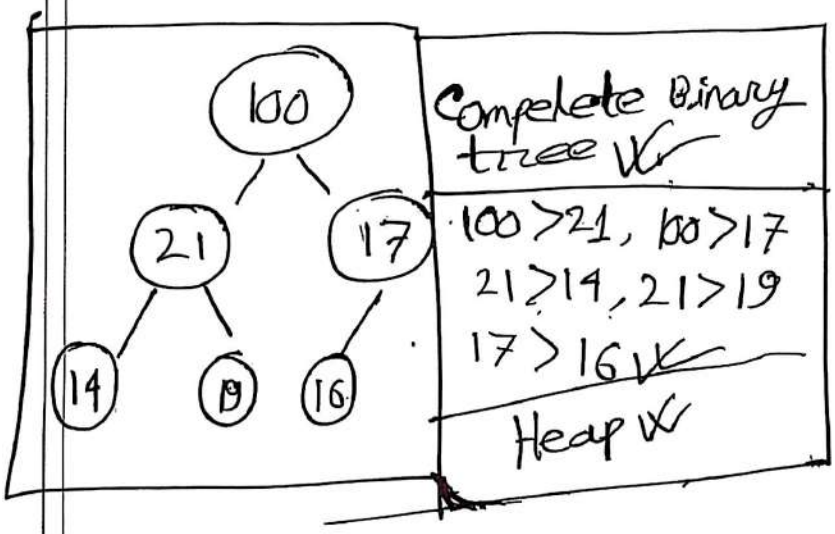


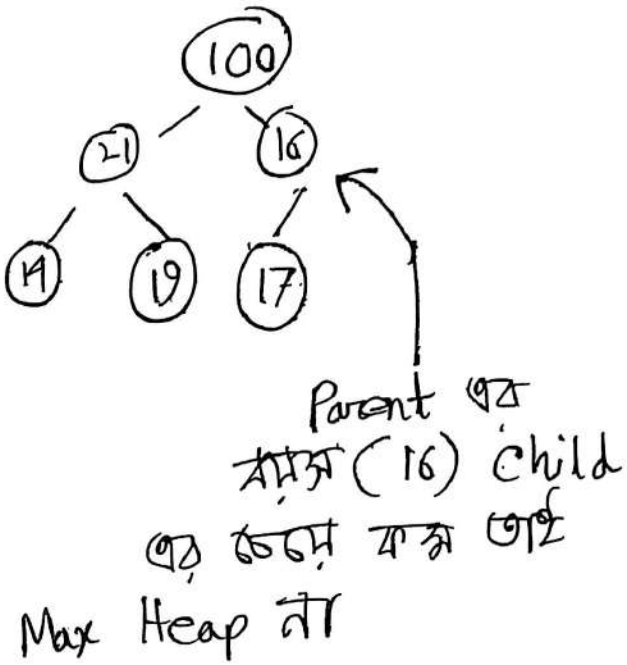
← Complete Binary tree



Heap হলো একটি Complete Binary tree

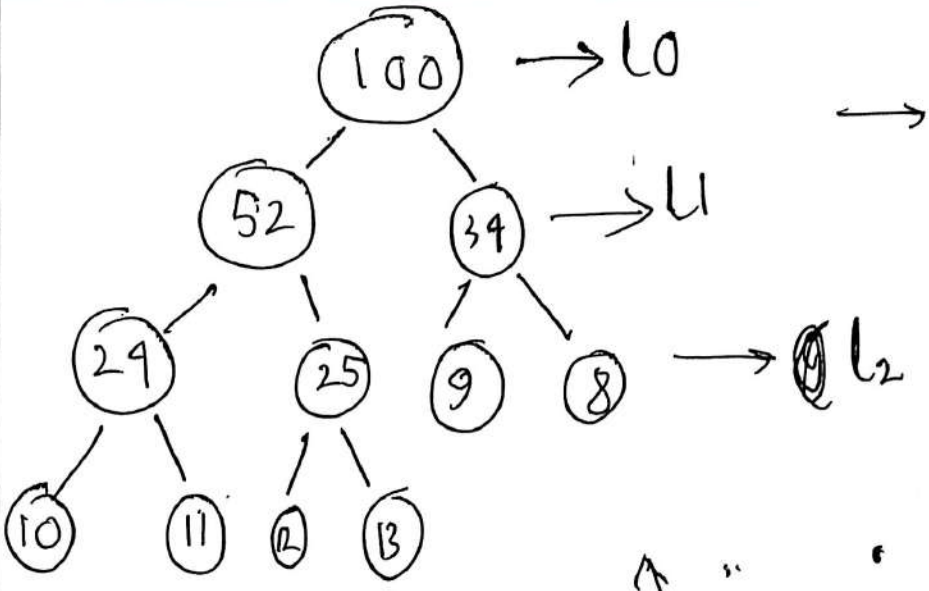
- Max heap —— যেখানে Level এ Parent এর Value child এর থেকে বড় বা সমান
- Min heap





Heap Operations

1. Insertion
2. DeleteMax or delete min
3. Make-heap / heapify

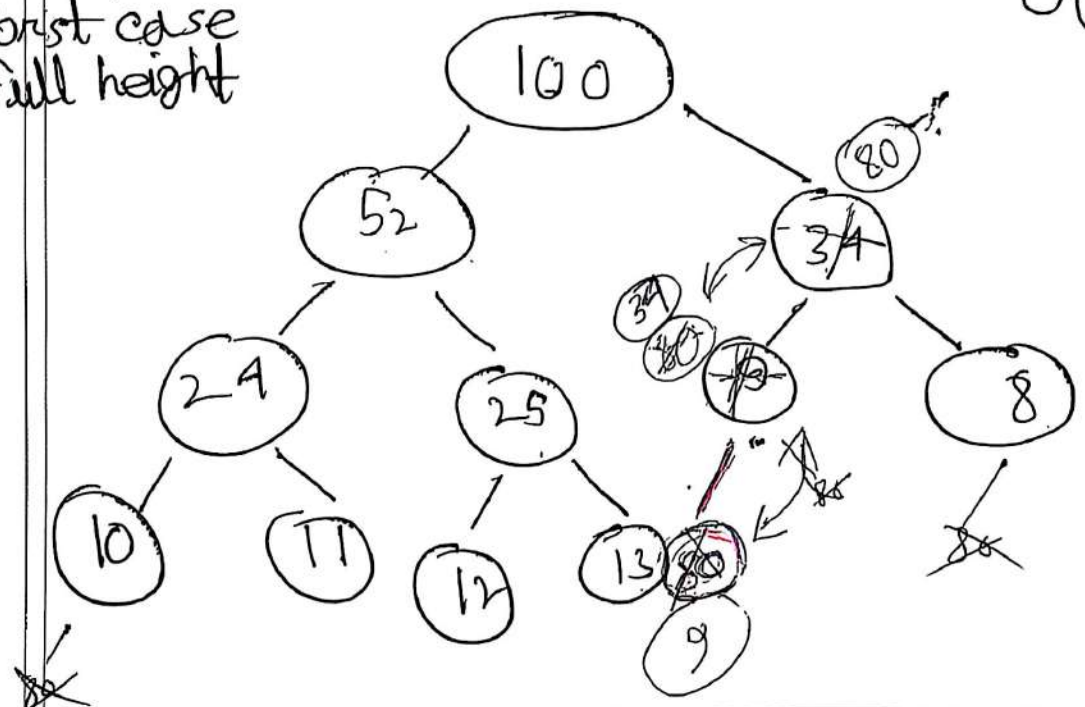


→ CBT ...

↑ Max-heapয়

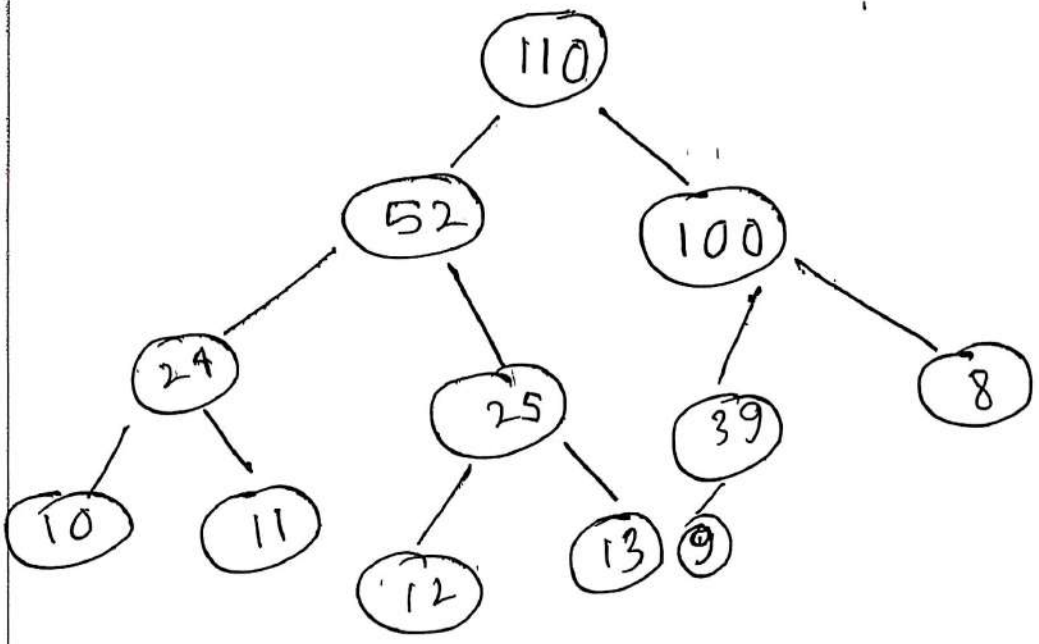
Insertion, ◦ Insert, delete এর time complexity $O(\log_2 n)$

Worst case এ full height



Tree related যেসব আসবে Algorithm লাগবে না। Simulation লাগবে।

Randomly যেকোন সাবট্রি থেকে Insert / Delete করা যায় না।



ডা

Heapify (Heap বানানো)

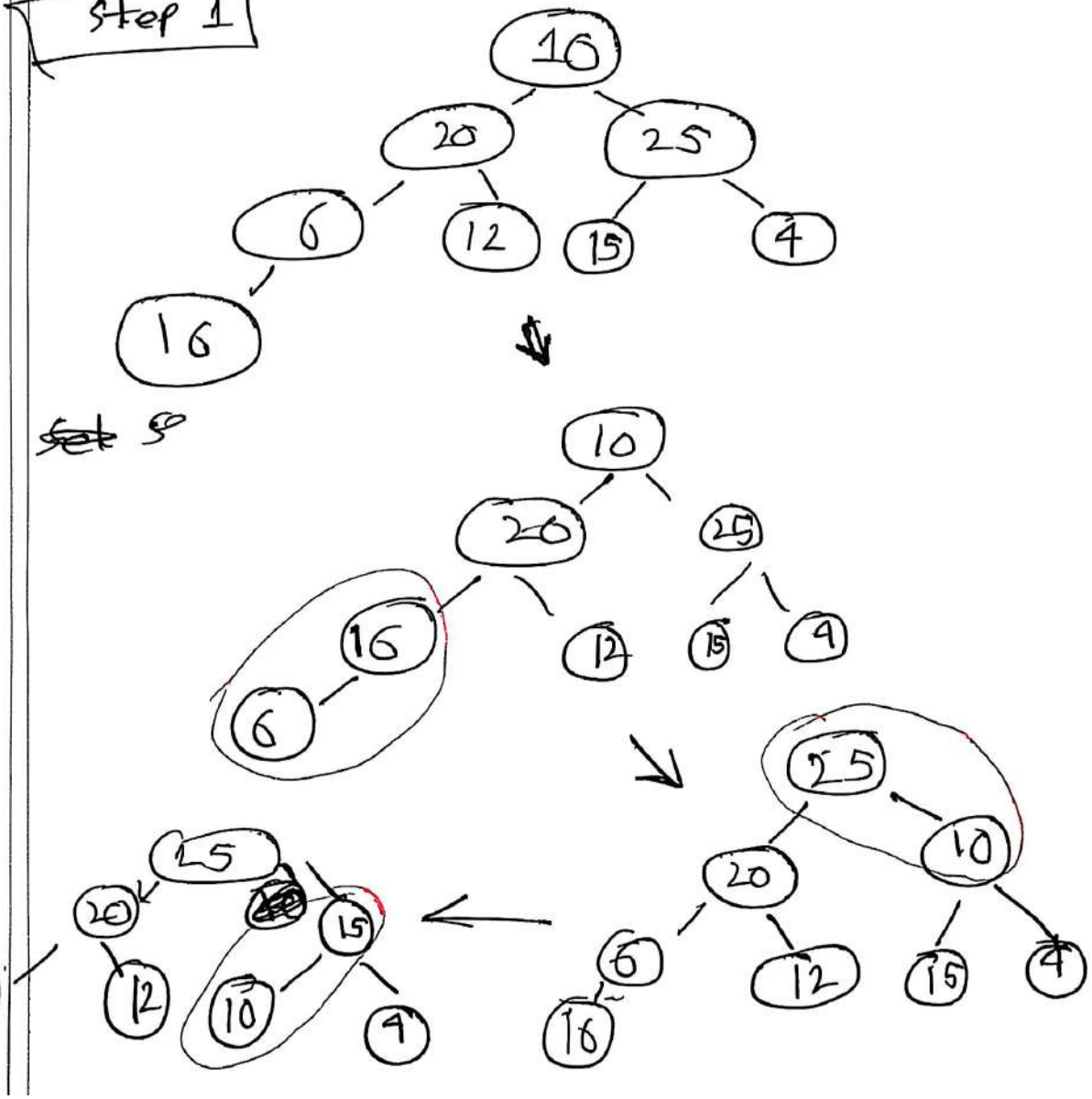
- Max heapify
- Min "

Step 1: যেই order এ আমাকে সেই order এ Heap বানাব

Step বাকিগুলো: swap - swap - swap - swap

10, 20, 25, 6, 12, 15, 4, 16

Step 1



Variant of Heap

1. Binary Heap
2. Binomial
3. Fibonacc
4. Pairing
5. Treap
6. Beap

Application

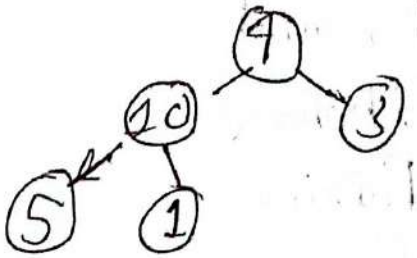
1. Heap Sort
2. Priority Q
3. Selection Algorithm
4. Graph algorithm

28/Oct/2026

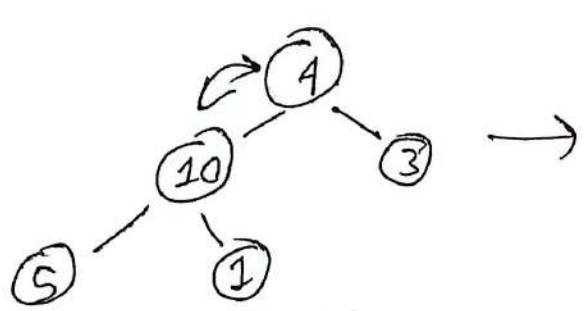
Heap.ppt (Slide 26)

Slide 27

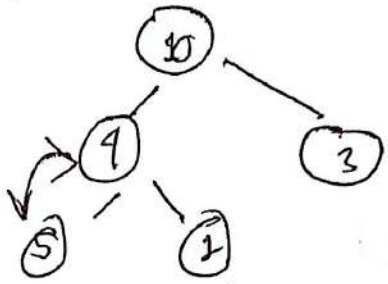
Arr = {4, 10, 3, 5, 1} →

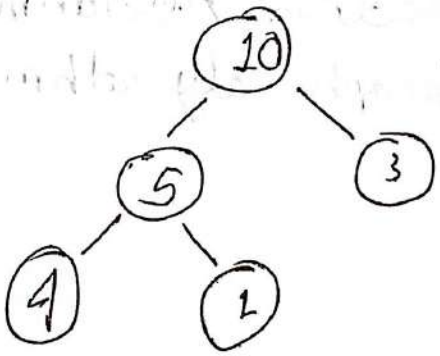


Transfer into max heap:



নিচের Level থেকে কোন swap প্রয়োজন হয় নাই

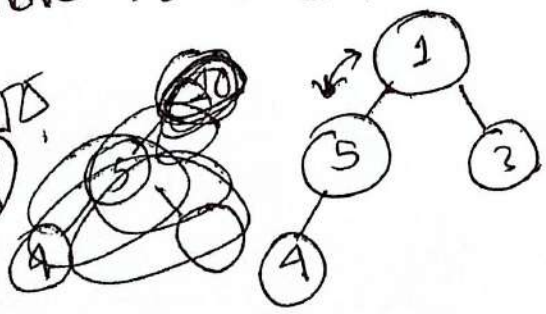


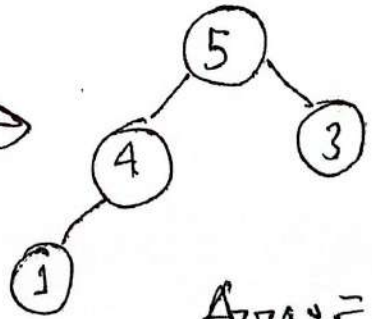


Slide 28

Remove Root (10)

(Right most element এর সাথে swap হবে)

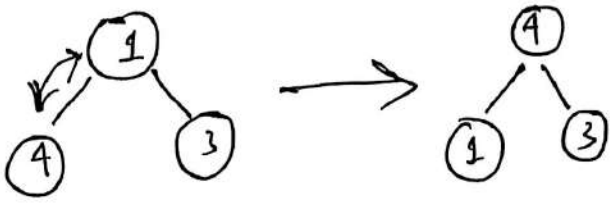




Array = {... 10}

Slide 30

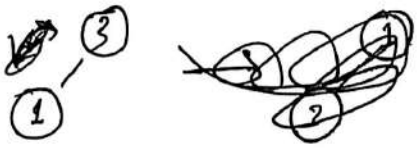
Remove Root (5)



Arr = {.... 5, 10}

~~Slide~~ Slide 31

Remove Root(4)



Arr = {...4, 5, 10}

~~Slide~~

Remove Root(3)



Arr = {..3, 4, 5, 10}

Arr = {1, 3, 4, 5, 10}

time complexity $O(n \times \log_2 n)$

Heap sort Traditional গুলার চেয়ে Faster. But Counting sort এর ক্ষেত্রে Range(k) না জেনে কোনটা Faster বলা যাচ্ছেনা।

Counting sort faster হবে যদি সেখানে ইনপুর ডিফি কম হয়।

Binary Search Tree (BST)

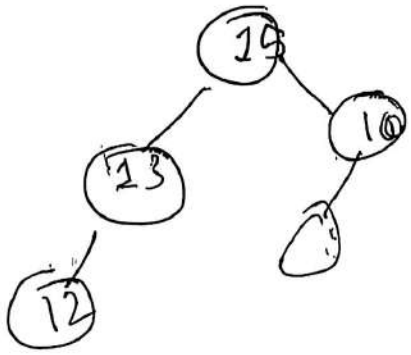
It is a hierarchical, nonlinear data structure

Left child, Parent এর চেয়ে ছোট হবে
Right child, Parent এর চেয়ে বড় বা সমান হবে

(সমান হলে একটু
পরিবর্তন
করতে পারেন)

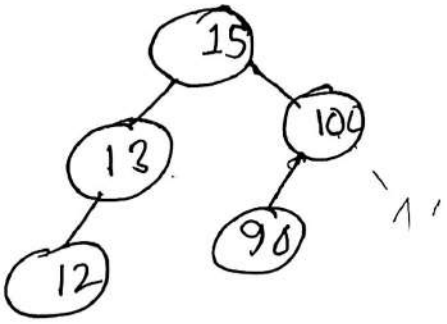
Search(Node, Key, Par)
1. If Node = NULL then return NULL
2. If Key < Node.key then search(Node, key, Node)

Tree এর কোন Algorithm লাগবে না সেহেতু লাগে পড়ার নাই।

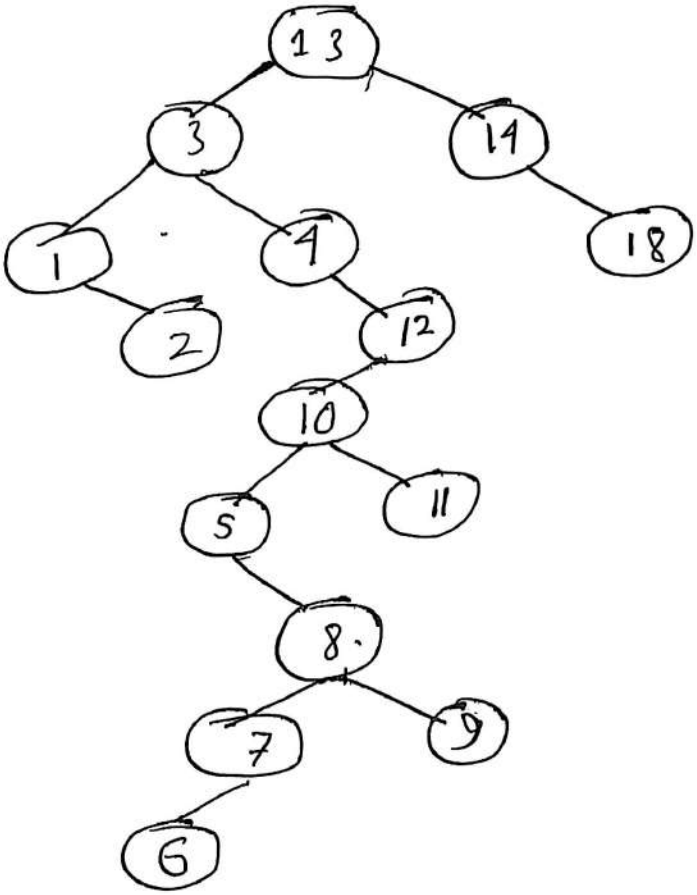


Insert

Insert (110)



- 13, 3, 4, 12, 14, 10, 5, 1, 8, 2, 7, 9, 11, 6, 18 in that order, create a BST starting from an empty tree

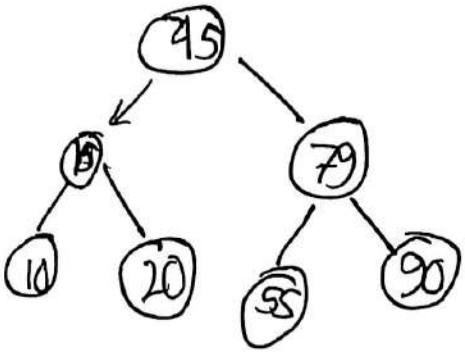


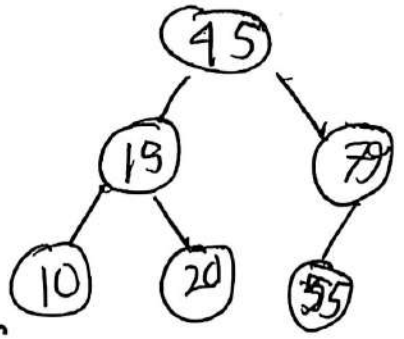
29/10/26

~~Inorder~~

Deleting a node from Binary search tree

delete(90)



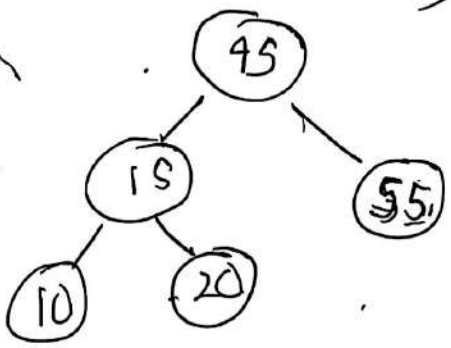


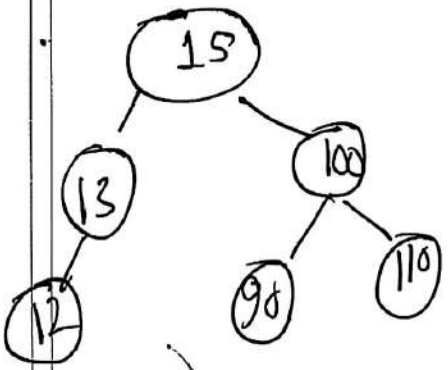
delete(45) next page

delete(79)

(Inorder successor)

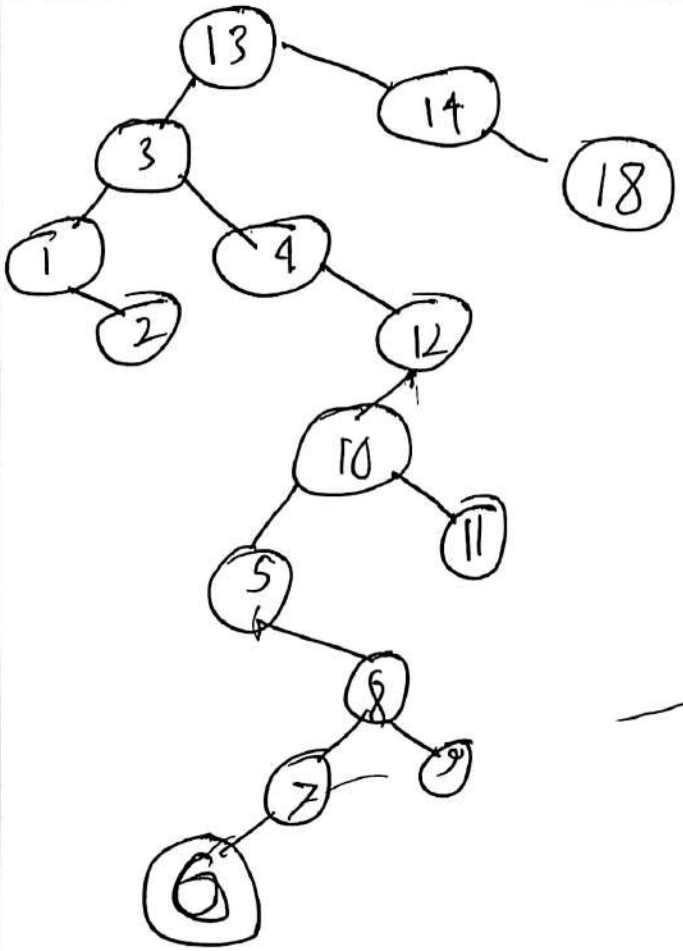
(ইনঅর্ডার সাকসেসর)



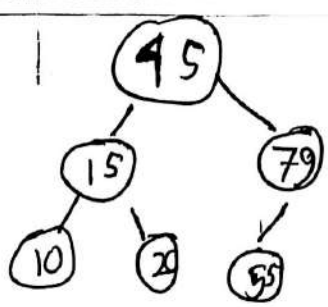


Inorder traversal : 12, 13, 15, 90, 100, 110

(Left → Root → Right)



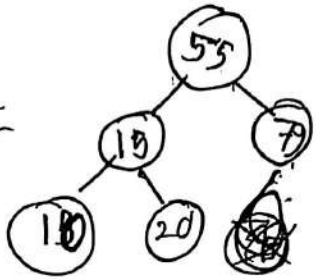
~~Inorder: 6, 7, 8, 9,~~



delete (45)

Inorder : 10, 15, 20, 45, 55, 79

পূর্ববর্তী পরবর্তী



Inorder: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 18,

(Ascending order এ আসা)

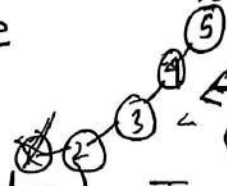
Time Complexity: $O(\log_2 n)$

↑

Insert, delete average case

delete worst case

example $O(n)$



← skewed tree

(সবই বাম সাইডে চলে গেছে)

BST

Insert, delete, Sort(Inorder), Time complexity

Tree এর Last topic

Huffman Algorithm

1952 সালে দিয়ে করেছে Algorithm. (Compression)

Application:

- File compression (ZIP, GZIP)
- Image formats (JPEG, PNG)
- MP3 audio compression
- Network protocols

Slide 7

Example

apple pie

Weight

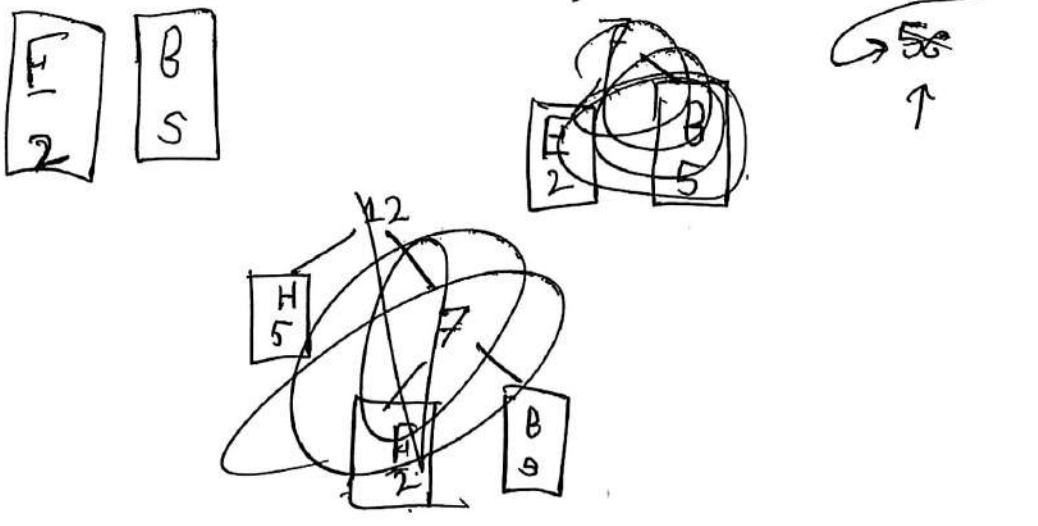
a : 1
p : 3
l : 1
e : 2
i : 1
space → □ : 1

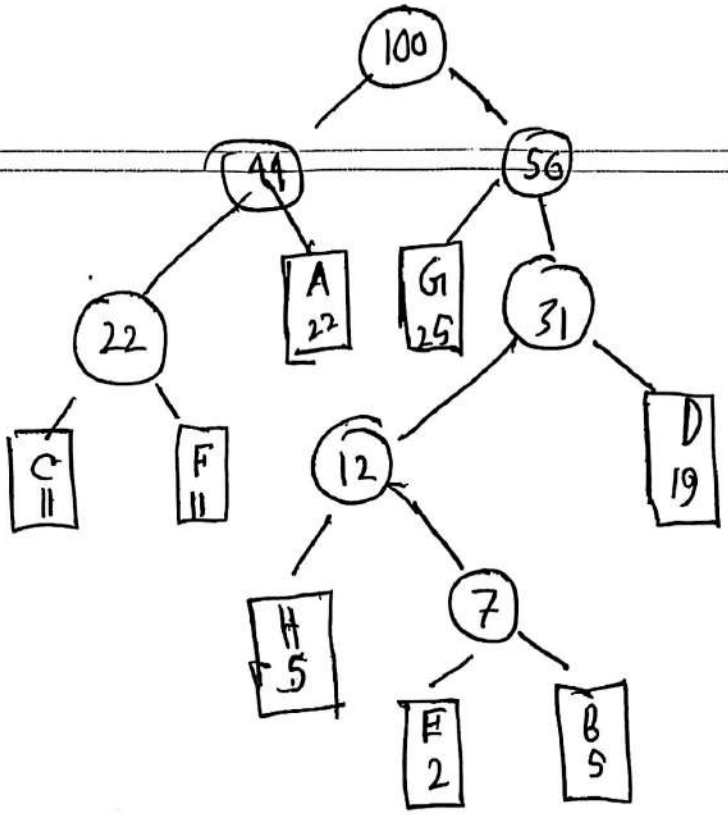
Given

A	B	C	D	E	F	G	H
22	5	11	19	2	11	25	5

প্রথমে Ascending আকারে সাজাতে হবে

F	B	H	C	F	D	A	G
2	5	5	11	11	19	22	25




100
44
56
A
22
G
25
22
31
C
11
F
11
12
D
19
H
5
7
E
2
B
5